Compilation, persistance et packaging sous Linux

Objectif du projet :

Ce projet illustre un pipeline complet sous Linux : écriture d'un programme C, compilation avec GCC, persistance via un service Systemd, et empaquetage dans un .deb. Il démontre directement un vecteur **supply chain** référencé dans le framework MITRE ATT&CK sous la technique **T1543.002** (*Create or Modify System Process: Systemd Service*).

Ce que fait le programme

- Tourne indéfiniment en boucle infinie (while(1))
- Affiche un message toutes les 3 secondes dans la sortie standard
- Utilise fflush(stdout) pour forcer l'écriture immédiate — indispensable pour que systemd capte les logs

Prérequis et structure du dépôt

Environnement : Debian / Ubuntu avec gcc, systemd, dpkg-deb installés.

```
cyber-monitor-daemon/
├── surveillance.c                          ← programme C source
├── surveillance                            ← binaire compilé
├── surveillance.service                    ← fichier service systemd
├── paquet-surveillance/
│   ├── DEBIAN/
│   │   ├── control                         ← carte d'identité du paquet
│   │   └── postinst                        ← script exécuté en root à l'install
│   ├── usr/local/bin/surveillance
│   └── etc/systemd/system/surveillance.service
└── README.md
```

Job 1 — Compilation GCC

Le programme surveillance.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    while (1) {
        printf("Le paquet fonctionne\n");
        fflush(stdout);
        sleep(3);
    }
    return 0;
}
```

Les 4 étapes de la chaîne GCC

Étape	Commande	Fichier produit	Ce qui se passe
1 — Préprocesseur	`gcc -E surveillance.c -o surveillance.i`	`.i`	Les `#include` sont remplacés par le contenu réel des headers
2 — Assembleur	`gcc -S surveillance.i -o surveillance.s`	`.s`	Le C est traduit en instructions assembleur (`mov` , `call` , `ret` ...)
3 — Fichier objet	`gcc -c surveillance.s -o surveillance.o`	`.o`	Code binaire, mais pas encore exécutable : `printf` et `sleep` ne sont pas encore résolus
4 — Édition de liens	`gcc surveillance.o -o surveillance`	`surveillance`	Les fonctions de la `libc` sont liées au programme → exécutable final

Pourquoi le .o n'est pas exécutable ? Il manque le code de printf et sleep, qui vivent dans la bibliothèque système libc. L'édition de liens les "colle" au programme.

Job 2 — Persistance via Systemd

1. Copier le binaire

```
sudo cp surveillance /usr/local/bin/
sudo chmod +x /usr/local/bin/surveillance
```

2. Créer le fichier service

Créer /etc/systemd/system/surveillance.service :

```
[Unit]
Description=Daemon de surveillance
After=network.target

[Service]
ExecStart=/usr/local/bin/surveillance
Restart=always
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Directive	Rôle
`After=network.target`	Attend que le réseau soit prêt au démarrage
`Restart=always`	**Persistance** : relance automatiquement si le process s'arrête
`StandardOutput=journal`	Redirige les logs vers journald
`WantedBy=multi-user.target`	Démarre au boot en mode normal

3. Activer et démarrer le service

```
systemctl daemon-reload
systemctl enable surveillance.service
systemctl start surveillance.service
```

4. Réflexion SOC — Détection

Un analyste SOC utilise journalctl pour surveiller les logs en temps réel :

```
journalctl -u surveillance.service -f
```

- -u : filtre par unité (le service ciblé)
- -f : suit en direct (équivalent tail -f)

Job 3 — Paquet Debian (.deb)

1. Créer l'arborescence

```
mkdir -p paquet/DEBIAN
mkdir -p paquet/usr/local/bin
mkdir -p paquet/etc/systemd/system

cp surveillance paquet/usr/local/bin/
cp /etc/systemd/system/surveillance.service
paquet/etc/systemd/system/
```

2. Fichier DEBIAN/control

```
nano /paquet/DEBIAN/control

Package: surveillance
Version: 1.0
Architecture: amd64
Maintainer: Nicolas
Description: Un daemon de surveillance en
persistance avec systemd
```

3. Script DEBIAN/postinst

Ce script s'exécute **automatiquement en root** à l'installation :

```
nano paquet/DEBIAN/postinst

  #!/bin/sh
  set -e

  systemctl daemon-reload
  systemctl enable surveillance.service
  systemctl start surveillance.service

  echo "Installation réussie : surveillance est
  actif."


chmod +x paquet-surveillance/DEBIAN/postinst
```

4. Compiler le paquet

```
dpkg-deb --build paquet-surveillance
dpkg -i paquet-surveillance.deb
```

Génère le fichier paquet-surveillance.deb

Étape 4 — Test et validation

Vérifier que le service tourne tout seul

```
sudo dpkg -i paquet-surveillance.deb
systemctl status surveillance.service
```

Résultat attendu : active (running) — **sans aucune action manuelle**. C'est la preuve que le vecteur supply chain fonctionne.

Réflexion MITRE ATT&CK — T1543.002

La technique T1543.002 (*Create or Modify System Process: Systemd Service*) est utilisée par des attaquants réels pour maintenir un accès persistant sur une machine Linux compromise.

Ce que ce projet démontre

Étape attaquant	Ce qu'on a fait
Compiler un binaire malveillant	`surveillance.c` compilé avec GCC
Installer la persistance	Service systemd avec `Restart=always`
Distribuer comme paquet légitime	`.deb` avec `postinst` auto-exécuté en root
Survivre au reboot	`WantedBy=multi-user.target` + `enable`

Comment un SOC détecte cette activité ?

- Surveillance des nouveaux fichiers dans /etc/systemd/system/
- Analyse des logs avec journalctl par unité
- Détection de paquets .deb non signés installés sur le système
- Audit des scripts postinst exécutés en root